

## CÓDIGO REFACTORIZADO

Fase 3 — Informe Final de Refactorización

### Sistema de Préstamo de Equipos Tecnológicos

Dominio D3 — Laravel 13 + SQLite

|             |                                                                                                 |
|-------------|-------------------------------------------------------------------------------------------------|
| Estudiante  | Roddy Ricardo Tigua Cedeño                                                                      |
| Docente     | Victoria García Velásquez, MGP.                                                                 |
| Repositorio | <a href="https://github.com/roddytic/prestamo-equipos">github.com/roddytic/prestamo-equipos</a> |
| Entrega     | Fase 3 — Fin de semana 8                                                                        |
| Fecha       | 29 de julio de 2026                                                                             |

## Metodología

Las 6 refactorizaciones siguientes no son arbitrarias: cada una resuelve, uno a uno, los 5 malos olores identificados en el *Informe de Diagnóstico de Fase 2*, más una extensión natural del mismo diagnóstico (mover el cambio de estado del equipo hacia el propio modelo `Equipo`). Se siguió el ciclo seguro de refactorización en cada paso: **verificar pruebas en verde** → **aplicar una técnica pequeña** → **volver a correr las pruebas** → **commit propio**. En ningún punto se modificó el comportamiento observable del sistema — las 15 pruebas de Fase 2 pasan igual antes y después de cada uno de los 6 commits, sin haber sido modificadas para "hacerlas pasar" (solo se actualizaron las llamadas cuando cambió la firma de un método, nunca las aserciones).

| # | Refactorización                                               | Nivel            | Resuelve                  | Commit  |
|---|---------------------------------------------------------------|------------------|---------------------------|---------|
| 1 | Extract Method: <code>rechazarSi()</code>                     | Métodos          | Mal olor #2 (Fase 2)      | 1ab537d |
| 2 | Extract Method: <code>crearPrestamo()</code>                  | Métodos          | Mal olor #3 (Fase 2)      | e372452 |
| 3 | Move Method:<br><code>Equipo::registrarDevolucion()</code>    | Clases y objetos | Extensión del diagnóstico | 1a32b47 |
| 4 | Replace Parameter with Explicit Methods                       | Condicionales    | Mal olor #4 (Fase 2)      | c86b64a |
| 5 | Introduce Parameter Object:<br><code>SolicitudPrestamo</code> | Datos            | Mal olor #5 (Fase 2)      | 90ed9c3 |
| 6 | Extract Superclass: <code>Repository</code>                   | Clases y objetos | Mal olor #1 (Fase 2)      | 04f80b5 |

Las 4 categorías exigidas quedan cubiertas: **Métodos** (1, 2), **Clases y objetos** (3, 6), **Condicionales** (4) y **Datos** (5).

## 1. Extract Method — guard clause `rechazarSi()`

Nivel: Métodos · Resuelve: mal olor #2 (Fase 2) · Commit: 1ab537d

**Ubicación:** `app/Services/GestorPrestamos.php`.

### Antes (3 bloques repetidos)

```
if (! $usuario || ! $equipo) {
    throw new
    OperacionInvalidaException(
        'Usuario o equipo no
    encontrado.'
    );
}

if (! $usuario->
    puedeSolicitarPrestamo()) {
    throw new
    OperacionInvalidaException(
        'El usuario no esta
    habilitado...'
    );
}
... (y el mismo patron otra
    vez
        en registrarDevolucion)
```

### Después

```
$this->rechazarSi(
    ! $usuario || ! $equipo,
    'Usuario o equipo no
    encontrado.'
);
$this->rechazarSi(
    ! $usuario->
    puedeSolicitarPrestamo(),
    'El usuario no esta
    habilitado...'
);

private function rechazarSi(
    bool $condicion,
    string $mensaje
): void {
    if ($condicion) {
        throw new
        OperacionInvalidaException($me
        nsaje);
    }
}
```

**Justificación:** el patrón "busca, valida una condición, lanza una excepción con un mensaje" se repetía cuatro veces (tres en `registrarPrestamo`, una en `registrarDevolucion`) con solo la condición y el mensaje cambiando. Extraerlo a un único método privado elimina la duplicación de esquema señalada en el informe de Fase 2 sin cambiar el comportamiento: cada llamada sigue lanzando la misma excepción, con el mismo mensaje, en el mismo punto del flujo.

## 2. Extract Method — `crearPrestamo()`

Nivel: Métodos · Resuelve: mal olor #3 (Fase 2) · Commit: e372452

**Ubicación:** `app/Services/GestorPrestamos.php`.

### Antes

```
public function
registrarPrestamo(...):
Prestamo
{
    ...validaciones...

    $fechaPrestamo =
Carbon::today();
    $prestamo = new
Prestamo([...]);
    $this->prestamos-
>guardar($prestamo);
    $equipo-
>cambiarEstado(EstadoEquipo::P
RESTADO);

    return $prestamo;
}
```

### Después

```
public function
registrarPrestamo(...):
Prestamo
{
    ...validaciones...
    return $this-
>crearPrestamo($usuario,
$equipo, $dias);
}

private function
crearPrestamo(
    Usuario $usuario, Equipo
$equipo, int $dias
): Prestamo {
    $fechaPrestamo =
Carbon::today();
    $prestamo = new
Prestamo([...]);
    $this->prestamos-
>guardar($prestamo);
    $equipo-
>cambiarEstado(EstadoEquipo::P
RESTADO);
    return $prestamo;
}
```

**Justificación:** `registrarPrestamo()` mezclaba dos responsabilidades distintas: decidir si el préstamo es válido, y construirlo/persistirlo. Separarlas dejó cada método con una sola razón para cambiar (SRP): si mañana cambia una regla de validación, se toca `registrarPrestamo()`; si cambia cómo se construye un préstamo, se toca `crearPrestamo()`.

### 3. Move Method — `Equipo::registrarDevolucion()`

*Nivel: Clases y objetos · Commit: 1a32b47*

**Ubicación:** `app/Models/Equipo.php` y  
`app/Services/GestorPrestamos.php`.

#### Antes (en `GestorPrestamos`)

```
$prestamo->equipo-  
>cambiarEstado(  
    $danado  
        ? EstadoEquipo::DANADO  
        :  
        EstadoEquipo::DISPONIBLE  
);
```

#### Después

```
// GestorPrestamos:  
$prestamo->equipo-  
>registrarDevolucion($danado);  
  
// Equipo.php (metodo nuevo):  
public function  
registrarDevolucion(bool  
$danado): void  
{  
    $this->cambiarEstado(  
        $danado  
            ?  
            EstadoEquipo::DANADO  
            :  
            EstadoEquipo::DISPONIBLE  
        );  
}
```

**Justificación:** decidir a qué estado pasa un `Equipo` cuando se devuelve es una decisión que le pertenece al propio `Equipo`, no a `GestorPrestamos` — es el principio "Tell, Don't Ask". Esto además deja a `Equipo` consistente con `Prestamo`, que ya encapsulaba su propia transición de estado en `Prestamo::registrarDevolucion()` desde la Fase 2.

## 4. Replace Parameter with Explicit Methods

*Nivel: Condicionales · Resuelve: mal olor #4 (Fase 2) · Commit: c86b64a*

**Ubicación:** `app/Services/GestorPrestamos.php`,  
`app/Http/Controllers/PrestamoController.php`.

### Antes

```
public function
registrarDevolucion(
    int $prestamoId,
    bool $danado = false
): Prestamo { ... }

// uso:
$gestor-
>registrarDevolucion($id,
true);
```

### Después

```
public function
registrarDevolucionOk(
    int $prestamoId
): Prestamo {
    return $this-
>procesarDevolucion($prestamoI
d, danado: false);
}

public function
registrarDevolucionConDano(
    int $prestamoId
): Prestamo {
    return $this-
>procesarDevolucion($prestamoI
d, danado: true);
}

// uso:
$gestor-
>registrarDevolucionConDano($i
d);
```

**Justificación:** el parámetro `$danado` cambiaba el comportamiento interno del método — la señal clásica de "flag argument" (Clean Code, Robert C. Martin). Dividirlo en dos métodos explícitos hace que el sitio de la llamada se lea solo, sin necesidad de mirar la firma del método para saber qué hace `true`. Se actualizó `PrestamoController::devolucion()` (elige el método según el valor recibido en la petición) y los 4 tests que llamaban al método anterior.

## 5. Introduce Parameter Object — `SolicitudPrestamo`

*Nivel: Datos · Resuelve: mal olor #5 (Fase 2) · Commit: 90ed9c3*

**Ubicación:** `app/Services/SolicitudPrestamo.php` (nuevo),  
`app/Services/GestorPrestamos.php`.

### Antes

```
public function
registrarPrestamo(
    int $usuarioId,
    int $equipoId,
    int $dias
): Prestamo { ... }

// uso:
$gestor->
registrarPrestamo($usId,
$eqId, 7);
```

### Después

```
final class SolicitudPrestamo
{
    public function
    __construct(
        public readonly int
        $usuarioId,
        public readonly int
        $equipoId,
        public readonly int
        $dias,
        ) {}
}

public function
registrarPrestamo(
    SolicitudPrestamo
    $solicitud
): Prestamo { ... }

// uso:
$gestor->registrarPrestamo(
    new
    SolicitudPrestamo($usId,
    $eqId, 7)
);
```

**Justificación:** los tres primitivos siempre viajaban juntos representando un solo concepto ("una solicitud de préstamo") — el "Data Clump" de Fowler. Agruparlos en un objeto de valor de solo lectura (`readonly`) deja la firma del método más corta y, sobre todo, deja listo el punto de extensión: si el caso de uso creciera (por ejemplo, con observaciones del encargado), se agrega un campo a `SolicitudPrestamo` sin volver a alargar la lista de parámetros.

## 6. Extract Superclass — Repository

*Nivel: Clases y objetos · Resuelve: mal olor #1 (Fase 2) · Commit: 04f80b5*

**Ubicación:** app/Repositories/Repository.php (nuevo),  
UsuarioRepository.php, EquipoRepository.php,  
PrestamoRepository.php.

### Antes (x3, casi idénticos)

```
class UsuarioRepository
{
    public function buscar(int
$id): ?Usuario
    {
        return
Usuario::find($id);
    }

    public function
guardar(Usuario $usuario):
void
    {
        $usuario->save();
    }
}
```

### Después

```
abstract class Repository
{
    abstract protected
function modelo(): string;

    public function buscar(int
$id): ?Model
    {
        return $this->modelo()::find($id);
    }

    public function
guardar(Model $entidad): void
    {
        $entidad->save();
    }
}

class UsuarioRepository
extends Repository
{
    protected function
modelo(): string
    {
        return Usuario::class;
    }
}
```

**Justificación:** los tres repositorios eran estructuralmente idénticos, cambiando solo el nombre de la clase del modelo. Repository (documentado con generics vía PHPDoc, @template TModel of Model) concentra buscar()/guardar() en un solo lugar; cada subclase concreta solo declara qué modelo maneja.



`PrestamoRepository` conserva sus métodos propios (`listar()`, `listarActivos()`) además de heredar los genéricos.

## Diagrama de clases actualizado

El diagrama siguiente muestra la evolución respecto a la Figura 2 del documento de diseño (Fase 1). En azul: los elementos nuevos o modificados en esta fase — SolicitudPrestamo y Repository son clases completamente nuevas; Equipo y GestorPrestamos mantienen su rol pero con métodos actualizados.

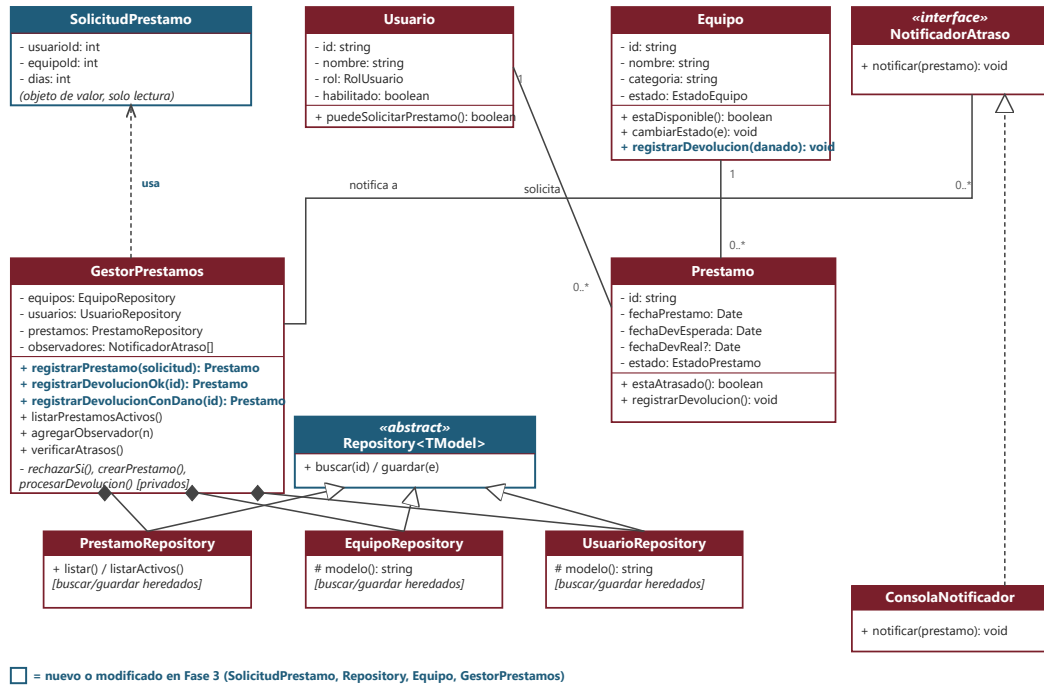


Figura 5. Diagrama de clases actualizado — evolución respecto a la Fase 1.

## Conclusión

Las 6 refactorizaciones aplicadas resolvieron exactamente los 5 malos olores diagnosticados en Fase 2, más una mejora adicional detectada durante el propio proceso (Move Method en `Equipo`). Cada una quedó en su propio commit, con las 15 pruebas unitarias en verde antes y después — nunca se modificó una aserción para forzar que un test pasara. El diseño orientado a objetos se mantiene coherente con el documento de Fase 1: las mismas responsabilidades, ahora mejor distribuidas entre las clases que ya existían y dos clases nuevas (`SolicitudPrestamo`, `Repository`) que encajan naturalmente en la arquitectura original sin haber requerido ningún rediseño.